Fundamentals of Data Science
PROJECT
WIN PREDICTOR

ABSTRACT

This project presents the development of a **Live T20 Cricket Win Predictor** using a supervised machine learning approach. The primary objective is to dynamically calculate the winning probability for the chasing team in a T20 match, updating the forecast after every ball based on the given match situation. The model is built upon a **Random Forest Classifier**, trained on a comprehensive historical dataset of ball-by-ball T20 match records. Critical **stateful features** were engineered, including **Runs Left**, **Balls Left**, **Wickets Left**, and the **Required Run Rate (RRR)**. These features explicitly model the game's complexity, eliminating the data leakage issues present in simpler models. The resulting model demonstrated a robust and realistic prediction accuracy of **99.04%** on its test set, significantly outperforming heuristic or static methods. The final trained model and team encoder were deployed within an interactive **Streamlit** web application, transforming the complex machine learning output into a userfriendly, real-time prediction tool. This work confirms the effectiveness of state-of-the-art machine learning in capturing the dynamic momentum shifts inherent in T20 cricket.

INTRODUCTION

LIVE T20 CRICKET WIN PREDICTOR

The domain of sports analytics, particularly within the fast-paced and data-rich environment of Twenty20 (T20) cricket, offers fertile ground for applied machine learning research. T20 matches are characterized by rapid shifts in momentum and high unpredictability, making the ability to accurately forecast the outcome in real-time a significant challenge. The **problem addressed** by this project is the lack of a robust, dynamic prediction model that moves beyond static pre-match odds. Traditional models often fail to account for the minute-byminute impact of wickets falling, boundaries scored, or changes in the required run rate (RRR).

The **primary objective** of this research is to develop and implement a **Live T20 Cricket Win Predictor**. This tool uses historical, ball-by-ball data to calculate the probability of the chasing team winning, updating this forecast after every single delivery.

Our approach involves several critical steps:

1. **Data Curation and Cleaning:** Merging inconsistent ball-by-ball datasets with match summary files to correctly assign the final `winner` label to every data point.
2. **Stateful Feature Engineering:** Creating eight key features, such as **Runs Left**, **Balls Left**, and **Wickets Left**, which encapsulate the entire *state* of the game. This step resolves the common issue of **data leakage** found in simpler prediction models.
3. **Model Selection and Training:** Utilising a **Random Forest Classifier** to train a robust model capable of mapping the complex interactions between these game-state features to the final match outcome.
4. **Deployment:** Packaging the final model and team encoder into an interactive web application built with **Streamlit** for real-time demonstration.

This project delivers a highly accurate and functionally deployable model, contributing a data-driven framework for understanding the dynamic probability of success in T20 cricket.

TECHNIQUES & METHODOLOGY

PRIMARY TECHNIQUES

- Machine Learning (ML)
- Win Probability Prediction (WPP)
- Random Forest Classifier
- Data Leakage (Crucial point of your successful resolution)
- Feature Engineering
- Streamlit

METHODOLOGY

- Supervised Learning
- Classification Model
- T20 Cricket Analytics
- Stateful Features (Refers to Runs Left, Wickets Left, etc.)
- Required Run Rate (RRR)
- Model Deployment

DOMAIN & PROJECT TITLE

- Sports Analytics
- In-Play Prediction
- Real-Time Forecasting
- Python Programming
- Data Preprocessing

IMPLEMENTATION

DATA COLLECTION

The project utilised historical ball-by-ball data from multiple T20 cricket seasons, obtained from four separate CSV files. This required merging the core ball-by-ball datasets with a separate match summary file using the common match_id column to append the final match winner as the target label.

DATA PREPROCESSING

Data preprocessing focused on resolving inconsistencies across the merged files, such as standardising column names like matchId to match_id. A critical feature engineering step was performed on the 2nd innings data to calculate the **stateful features** (eg: runs_left, wickets_left, balls_left) for every single ball.

EXPLORATORY DATA ANALYSIS [EDA]

EDA was primarily focused on understanding the **distribution of match outcomes** based on key game-state variables like **Required Run Rate (RRR)** and **Wickets Left**. This step was essential for confirming the non-linear relationship between match momentum and the final result, validating the need for a robust model.

MODEL BUILDING

The model used is a **Random Forest Classifier**, selected for its ability to handle the complex, non-linear interactions between the stateful features and the binary win/loss outcome. This approach ensures the model learns the dynamic impact of events like a wicket or a boundary on the win probability.

TRAIN-TEST-SPLIT

The preprocessed, stateful dataset was partitioned into **80% for training** and **20% for testing** using a stratified split. This ensures that the test set accurately reflects the diversity of game situations present in the training data, providing a reliable measure of the model's generalization ability.

MODEL EVALUATION

Model evaluation was performed using **accuracy** as the primary metric, achieving a high score of **99.04%** on the unseen test data. This figure confirms that the model is effective at predicting match outcomes based on the live game state, successfully validating the entire feature engineering and training pipeline.

RESEARCH AND REFERENCES

1) https://link.springer.com/chapter/10.1007/978-3031-67871-4_11

2) https://www.sciencedirect.com/science/article/abs/pii/S0169207011001622?via%3Dihub

3) https://sathvik-nishanth.medium.com/rankingtheindian-premier-league-ipl-teams-using-theelorating-system-3bbfc2f14605

4) https://www.sciencedirect.com/science/article/abs/pii/S0169207015000618?via%3Dihub

5) https://sathvik-nishanth.medium.com/ranking-theindianpremier-league-ipl-teams-using-the-elo-ratingsystem3bbfc2f14605

6) https://link.springer.com/chapter/10.1007/978-981992468-4_29?fromPaywallRec=true

7) https://scholar.google.com/scholar?&q=Singh%2C%20T.%2C%20Singla%2C%20V.%2C%20Bhatia%2C%20P.%3A%20Score%20and%20winning%20prediction%20in%20cricket%20through%20data%20mining.%20In%3A%202015%20International%20Conference%20on%20Soft%20Computing%20Techniques%20and%20Implementations%20%28ICSCTI%29%2C%20pp.%2060%E2%80%9366.%20IEEE%20%282015%29

CONCLUSION

This project successfully developed and deployed a robust **Live T20 Cricket Win Predictor** using a **Random Forest Classifier**. By prioritizing **stateful feature engineering** (Runs Left, Balls Left, Wickets Left, RRR), the model overcame the critical challenge of data leakage prevalent in simpler predictive systems. The resulting model demonstrated exceptional predictive capability, achieving an accuracy of **99.04%** on unseen test data. The successful integration of this complex model with an interactive **Streamlit** application validates the feasibility of using advanced machine learning for real-time dynamic forecasting in sports analytics, providing immediate, data-driven insight into the ebb and flow of a T20 match.

FUTURE SCOPE

- **Advanced Feature Integration:** Incorporate contextual data such as the **Pitch Report/Venue History**, **Weather Conditions**, **Head-to-Head Team Records**, and the **Toss Decision** (which you planned to add) to provide more nuanced predictions.

- **Player-Specific Impact:** Integrate real-time player ratings (e.g., historical strike rates for the current batsman and economy rates for the current bowler) to model their specific, granular impact on probability.

- **Model Comparison:** Experiment with deep learning architectures, such as **Recurrent Neural Networks (RNNs)** or **Long Short-Term Memory (LSTM) models**, which are specifically designed to handle sequential time-series data, potentially capturing momentum shifts even more effectively than the current Random Forest model.

- **Bias Calibration:** Further analyze the instances where the model makes errors to identify any underlying biases (e.g., underestimating the chase probability when fewer wickets remain) and apply probability calibration techniques.